

TALLINN UNIVERSITY OF TECHNOLOGY  
IT College

Marko Linde 176292IADB

# **CALCULATOR**

Javascript (ICD0006)

Instructor: Andres Käver

Tallinn 2022

## Table of Contents

|                                                |   |
|------------------------------------------------|---|
| 1 Assignment .....                             | 3 |
| 2 Overview .....                               | 3 |
| 2.1 Calculator logic.....                      | 3 |
| 2.2 Display.....                               | 5 |
| 2.3 Functionality.....                         | 6 |
| 2.3.1 Main.....                                | 6 |
| 2.3.2 Evaluations after calculate command..... | 6 |
| 2.3.3 Calculation command.....                 | 6 |
| Annex 1 – index.js .....                       | 7 |

# 1 Assignment

Write simple calculator using Bootstrap (or any other css lib) based design and simple js functionality.

Provide basic functions - +, -, \*, /, C

Write your calculator using an object oriented approach. Separate code between business logic and ui.

Please implement classical calculator UI - like you have in your mobile phone.

Nice to have - calculator is done in responsive design. Fits full-screen, both on desktop and on mobile.

## 2 Overview

Index.html and index.css taken from calculator example . Index.js is written by document author.

### 2.1 Calculator logic

Basic idea comes from logic, that the functionality of the calculator can be expanded.

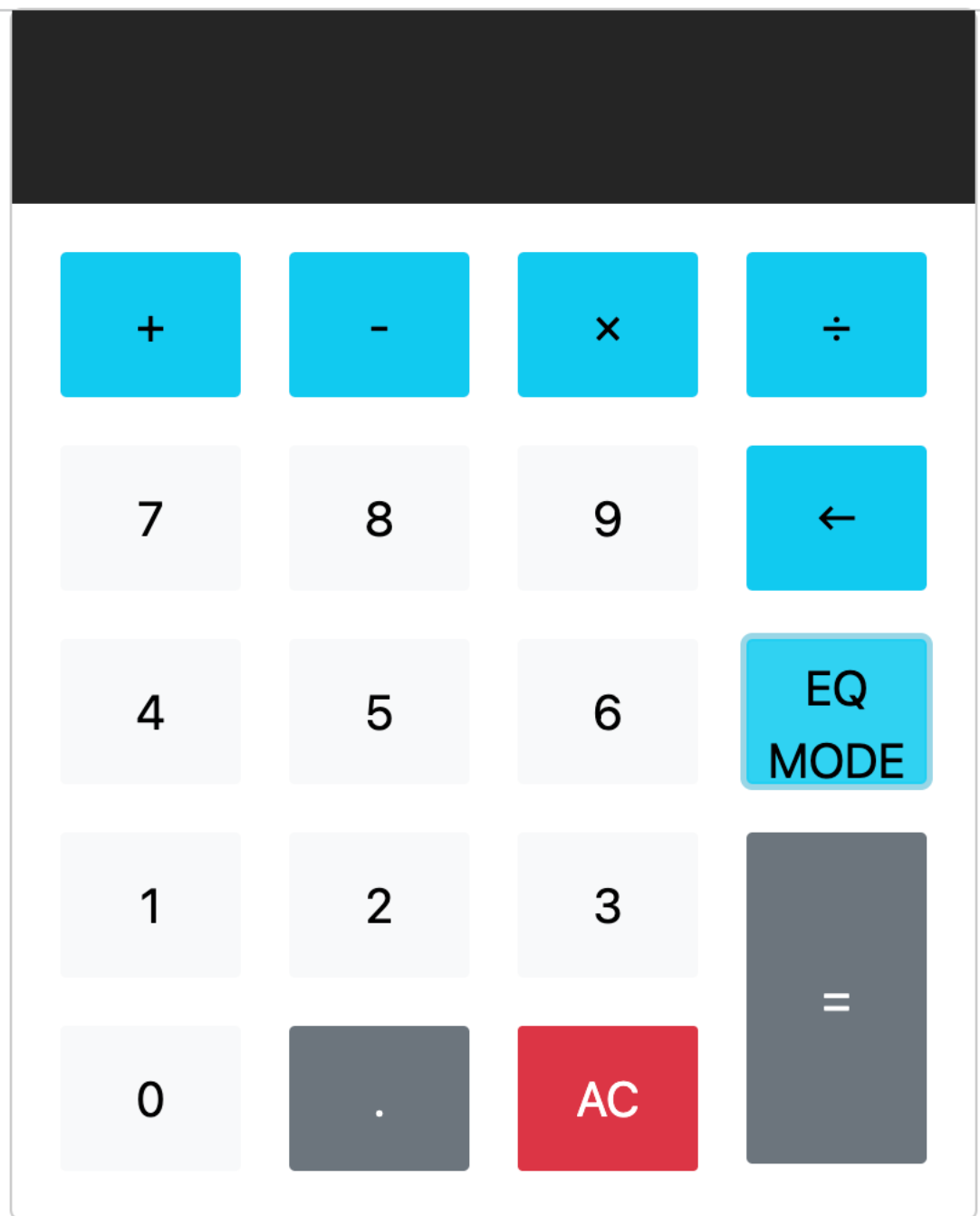
- All calculator clicks are saved in a long string only array which allows:
  - to display the entire equation to the user



Drawing 1. Equation view

- to add undo button functionality
  - to add calculate only parts of the equation functionality
  - to add click on equation element and repair it directly functionality
  - to make calculator more scientific
- Calculator starts to calculate only when clicking the equal sign
- In EQ mode the calculations are done from left to right with rule: first multiply or divide then add or subtract
- In Not EQ mode the calculations are done from left to right
- Backspace removes last equation entry

## 2.2 Display



Drawing 2. Calculator screen

## **2.3 Functionality**

### **2.3.1 Main**

- All buttons have class “btn” and are observed through it.
- All buttons have distinct values assigned to them and differentiated by those values
- Every button click value is inserted into equations array separately (needs to change at next iteration for evaluate right away functionality - is faster and simpler)
- Equal button clicking calls calculation function
- AC button click calls overwriting of the previous array by empty array
- Backspace removes last equation entry
- EQ mode switches between two calculation modes

### **2.3.2 Evaluations after calculate command**

- Operators the start (except minus) and end equation are evaluated – that they don't exist
- Two operators straight are not allowed
- Two dots straight are not allowed
- Operator and dot are not allowed

### **2.3.3 Calculation command**

- Number of operators is searched
- First calculation operator is searched from the equation and its index is returned.
- All numbers between operators are converted to full numbers
- Calculated value is inserted back into the equation
- New calculation command iteration will begin

## Annex 1 – index.js

```
"use strict";
import css from "./index.css";

/**
 * Equation is class to preserve user clicked
 * information and perform basic operations with
 * array equation
 */
class Equation {

    equation = [];

    constructor(){
        this.equation = [];
    }

    clear(){
        this.equation = [];
        this.number= [];
        console.log(JSON.stringify(this.equation));
    }

    addElement = (elem) => {
        this.equation.push(elem);
        console.log(JSON.stringify(this.equation));
    }
}
```

```

// all individual numeric elements between symbols are converted to full numbers
convertEquationToFullNumbers = (operators) =>{
  let index=0;
  let number="";
  let compressedEquation=[];

  this.equation.forEach(elem =>{
    if (this.getIndex(operators,elem) > -1 && !(elem === '-' && index === 0)){
      compressedEquation.push(number);
      compressedEquation.push(elem);
      number="";
    } else {
      if (number === "") number = elem;
      else number = number.concat(this.equation[index]);
    }
    index++;
  });
  if (number !== ""){
    compressedEquation.push(number);
  }

  this.equation = compressedEquation;
  console.log(this.equation);
}

getNumberOfSymbolsInEquation = (operators) =>{
  let counter = 0;

  this.equation.forEach(elem => {
    if (this.getIndex(operators,elem) > -1) counter +=1;
  });

  return counter;
}

```

```

getIndex(operators,sign){
    return operators.findIndex((element) => element === sign);
}

getConcatEquation = () =>{
    return this.equation.join('');
}

getIndexValue = (index) => {
    return this.equation[index];
}

getPreviousValue = (index) => {
    return this.equation[index-1];
}

getNextValue = (index) => {
    return this.equation[index+1];
}

changeEquation = (index,value) =>{
    if (index >= 0) this.equation.splice(index - 1,3,value);
}

removeLast = () =>{
    this.equation.pop();
}
}

/**
 * Selector class is html similar pointers perserver.
 */
class Selector{
    constructor(tag,selector){
        this.tag = tag;
        this.tribute = document.querySelectorAll(selector);
    }
}

```

```

/**
 * Takes care of calculator user interface, which
 * relays the information from web page to calculation
 * and pack
 */
class CalculatorUI{
  constructor(calculus){
    this.calculus = calculus;
    this.selectors = [];

    this.selectors.push(new Selector('button', '.btn'));
    this.selectors.push(new Selector('screen', '.calculator-screen'));

    //iterate through all monitored elements
    this.selectors.forEach(selector =>{
      if (selector.tag === 'button'){
        selector.attribute.forEach(elem =>{
          if (elem){
            elem.onclick = (() => {
              this.action(elem.getAttribute('value'));
              if (elem.getAttribute('value') === 'eqMode'){
                elem.style.color= (elem.style.color === 'red') ? 'black' : 'red';
              }
            })
          }
        })
      }
    });
  }

  action = (action) => {
    this.calculus.actionCaller(action);
    this.refresh();
  }
}

```

```

//refreshes display
refresh = () => {
  this.selectors.forEach(selector =>{
    if (selector.tag === 'screen'){
      selector.attribute.forEach(elem =>{
        if (elem){
          elem.value = this.calculus.getEquationState();
        }
      })
    }
  });
}
}

/**
 * Businesslogic class to perform calculations and find errors
 * Todo: move errors to separate class
 */
class Calculus {

  equation;
  equationState;
  operators;
  eqMode;

  constructor(){
    this.equation = new Equation();
    this.equationState = undefined;
    this.operators = ['+', '-', '*', '/'];
    this.eqMode = 0;
  }
}

```

```

actionCaller = (action) =>{
  console.log(action);
  console.log(JSON.stringify(this.equation));

  if (!(this.findErrorAction(action))){
    switch (action) {
      case 'all-clear':
        this.equation.clear();
        break;
      case '=':
        this.equation.convertEquationToFullNumbers(this.operators);
        this.calculate();
        break;
      case 'undo':
        this.equation.removeLast();
        break;
      case 'eqMode':
        this.eqMode = this.setEqMode();
        console.log("Eqmode :" + JSON.stringify(this.eqMode));
        break;
      default:
        this.equation.addElement(action);
    }
  }
  this.equationState = this.equation.getConcatEquation();
}

//switches system between mathematical standard and linear calculation modes
setEqMode = () =>{
  return (this.eqMode === 1) ? 0 : 1;
}

//returns equation to screen
getEquationState = () =>{
  return this.equationState;
}

```

```

//find user entered problems
findErrorAction = (action) =>{
    let error = 0;
    if (this.equation.getConcatEquation().length === 0 && this.equation.getIndex(this.operators,action) > -1 &&
        action !== '-') error = 1;
    else if ((this.isPreviousEntryAnOperator(this.operators) || this.isPreviousEntryAnOperator(['.'])) &&
        action === '=') error = 1;
    else if ((this.isPreviousEntryAnOperator(this.operators) || this.isPreviousEntryAnOperator(['.'])) &&
        (this.equation.getIndex(this.operators,action) > -1 || action === '.')) error = 1;
    return error;
}

isPreviousEntryAnOperator = (operators) =>{
    return (this.equation.getIndex(operators,
        this.equation.equation[this.equation.equation.length-1]) > -1) ? 1 : 0;
}

calculate = () =>{
    while(this.equation.getNumberOfSymbolsInEquation(this.operators)){
        if (this.eqMode === 1) this.calculateIndexValue(this.getNextCalculationIndex());
        else this.calculateIndexValue(1);
    }
}

```

```

//calculate equation value on index operator with before and after numbers
//and inserts subcalculation back to equation
calculateIndexValue = (startIndex) =>{
  let value = NaN;

  switch(this.equation.getIndexValue(startIndex)) {
    case '*':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) *
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    case '/':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) /
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    case '+':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) +
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    case '-':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) -
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    default:
      value;
  }
  console.log('Index calculation value: ' + value);
  this.equation.changeEquation(startIndex,value.toString());
  console.log('Equation: ' + JSON.stringify(this.equation.equation));
}

```

```

//based on mathematical logic get first calculation operator
getNextCalculationIndex = () =>{
    let startIndex = NaN;

    if (((this.getEquationIndex('*') !== -1) && (this.getEquationIndex('*') < this.getEquationIndex('/'))) ||
        ((this.getEquationIndex('/') === -1) && (this.getEquationIndex('*') > this.getEquationIndex('/'))))
        startIndex = this.getEquationIndex('*');
    else if (this.getEquationIndex('/') >= 1) startIndex = this.getEquationIndex('/');
    else if (this.getEquationIndex('+') >= 1) startIndex = this.getEquationIndex('+');
    else if (this.getEquationIndex('-') >= 1) startIndex = this.getEquationIndex('-');

    console.log('Starting with index: ' + startIndex);
    return startIndex;
}

getEquationIndex = (value) =>{
    return this.equation.getIndex(this.equation.equation,value);
}
}

/**
 * Calling calculation program
 */
new CalculatorUI(new Calculus());

```